LAPORAN PRAKTIKUM
POSTTEST (1)

PEMROGRAMAN BERORIENTASI OBJEK



Disusun oleh:

Muhammad Dava Fahriza

(2509106100)

C1'26

PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2025

1. Penjelasan program

Program ini dibuat untuk mengelola penjualan produk skincare menggunakan konsep OOP. Program memiliki tiga class, yaitu Produk untuk mengatur produk dan stok, Pelanggan untuk mengatur data pelanggan dan saldo, serta Penjualan untuk menghitung transaksi. Program juga menggunakan atribut public dan private, getter, setter, serta validasi agar data seperti stok, saldo, dan total tidak bisa diisi dengan nilai negatif. Selain itu, terdapat instance method, class method, dan static method yang digunakan untuk menjalankan berbagai fungsi dalam program.

2. Tujuan Program

- Menampilkan data produk dan stok.
- Mengelola data pelanggan dan saldo.
- Menghitung total penjualan.
- Mengubah data menggunakan setter.
- Melakukan validasi data.
- Menghitung diskon.
- Mengelola data umum toko.

3. Penjelasan Fungsi

1. Class Produk digunakan untuk menyimpan data produk skincare seperti nama, harga, dan stok. nama_toko, jenis_toko, dan jumlah_produk merupakan atribut kelas. Sedangkan nama, harga, dan __stok merupakan atribut object.



```
class Produk:
    nama_toko = "Glow Skincare"
    jenis_toko = "Skincare"
    jumlah_produk = 0

    def __init__(self, nama, harga, stok):
        self.nama = nama
        self.harga = harga

        self.__stok = stok

        Produk.jumlah_produk += 1
```

Gambar 3.1 Class Produk

2. Method tampilkan() digunakan untuk menampilkan informasi produk. Karena menggunakan self, method ini termasuk instance method.



```
def tampilkan(self):
        print("Nama  :", self.nama)
        print("Harga : Rp", self.harga)
        print("Stok  :", self.__stok)
```

Gambar 3.2 Instance Method

3. Getter digunakan untuk mengambil nilai stok, sedangkan setter digunakan untuk mengubah stok. Setter juga melakukan validasi agar stok tidak boleh bernilai negatif.

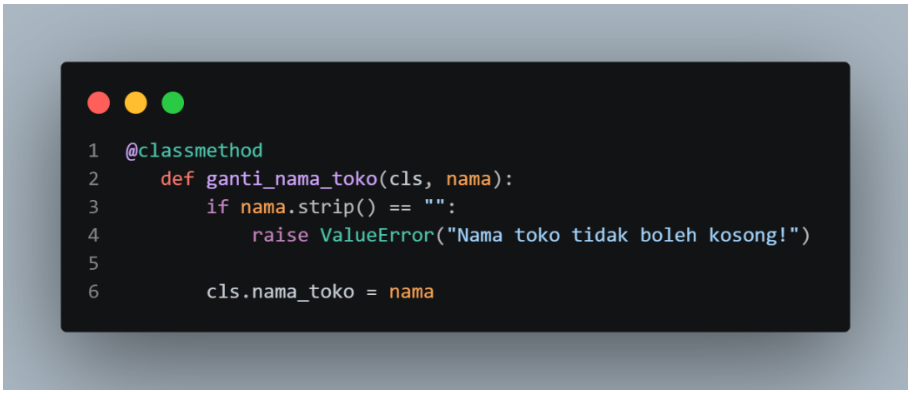


```
@property
  def stok(self):
      return self.__stok

  @stok.setter
  def stok(self, nilai):
      if nilai < 0:
          raise ValueError("Stok tidak boleh negatif!")
      self.__stok = nilai
```

Gambar 3.3 Getter dan Setter Stok

4. Method ini digunakan untuk mengubah nama toko. Karena menggunakan cls, method ini termasuk class method.



```
@classmethod
    def ganti_nama_toko(cls, nama):
        if nama.strip() == "":
            raise ValueError("Nama toko tidak boleh kosong!")

        cls.nama_toko = nama
```

Gambar 3.4 Class Method

5. Method ini digunakan untuk mengecek apakah harga lebih dari 0. Static method tidak menggunakan self atau cls.



```python
@staticmethod
    def cek_harga(harga):
        return harga > 0
```

Gambar 3.5 Static Method

6. Class Pelanggan digunakan untuk menyimpan data pelanggan berupa nama, alamat, dan saldo. Saldo dibuat private menggunakan __saldo.



```python
class Pelanggan:

    nama_toko = "Glow Skincare"
    jumlah_pelanggan = 0
    status_toko = "Aktif"

    def __init__(self, nama, alamat, saldo):
        self.nama = nama
        self.alamat = alamat
        self.__saldo = saldo

        Pelanggan.jumlah_pelanggan += 1
```

Gambar 3.6 Class Pelanggan

7. Getter mengambil saldo pelanggan, sedangkan setter mengubah saldo. Nilai negatif akan ditolak oleh program.



```python
@property
    def saldo(self):
        return self.__saldo

    @saldo.setter
    def saldo(self, nilai):
        if nilai < 0:
            raise ValueError("Saldo tidak boleh negatif!")

        self.__saldo = nilai
```

Gambar 3.7 Getter dan Setter Saldo

8. Class Penjualan digunakan untuk mengatur transaksi. Data yang disimpan adalah pelanggan, produk, jumlah pembelian, dan total harga.



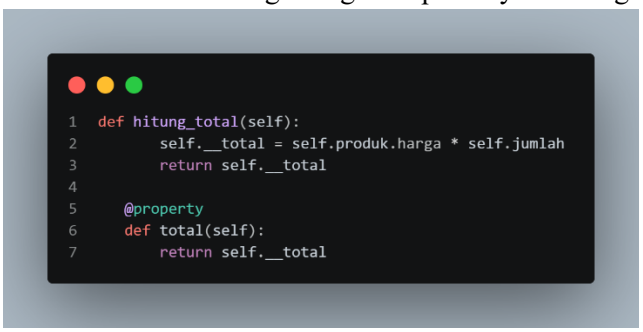
```python
class Penjualan:
    nama_toko = "Glow Skincare"
    jumlah_transaksi = 0
    pajak = 0.10

    def __init__(self, pelanggan, produk, jumlah):
        self.pelanggan = pelanggan
        self.produk = produk
        self.jumlah = jumlah
        self.__total = 0

        Penjualan.jumlah_transaksi += 1
```

Gambar 3.8 Class Penjualan

9. Method ini menghitung total pembayaran dengan rumus harga produk × jumlah pembelian.

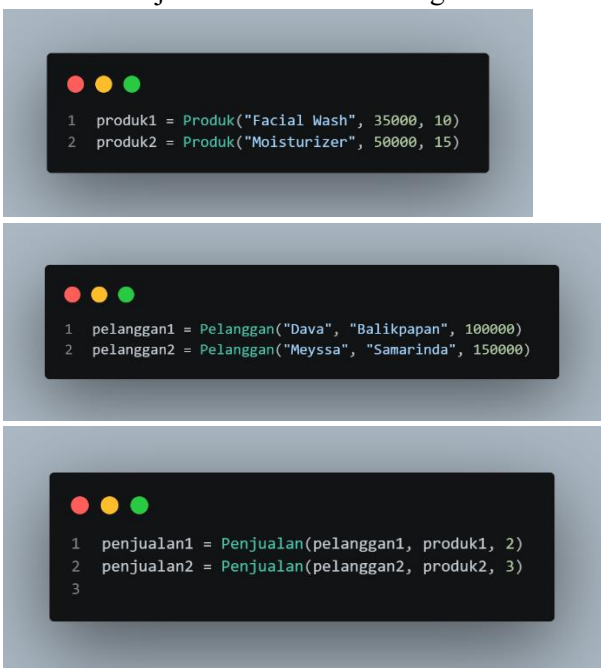


```python
def hitung_total(self):
        self.__total = self.produk.harga * self.jumlah
        return self.__total

    @property
    def total(self):
        return self.__total
```

Gambar 3.9 Menghitung Total

10. Bagian ini digunakan untuk membuat object dari setiap class. Masing-masing class dibuat minimal dua object sesuai ketentuan tugas.



```python
produk1 = Produk("Facial Wash", 35000, 10)
produk2 = Produk("Moisturizer", 50000, 15)
```

```python
pelanggan1 = Pelanggan("Dava", "Balikpapan", 100000)
pelanggan2 = Pelanggan("Meyssa", "Samarinda", 150000)
```

```python
penjualan1 = Penjualan(pelanggan1, produk1, 2)
penjualan2 = Penjualan(pelanggan2, produk2, 3)
```

Gambar 3.10 Membuat Objek

11. Program mencoba mengubah stok dengan nilai yang benar dan salah. Jika nilainya negatif, program menampilkan pesan bahwa stok tidak boleh negatif.



```
produk1.stok = 20

try:
    produk1.stok = -5
except ValueError as e:
    print("Validasi:", e)
```

Gambar 3.11 Pengujian Setter

12. Bagian ini digunakan untuk membuktikan bahwa instance method, class method, dan static method dapat berjalan dengan baik.



```
produk1.tampilkan()

print(penjualan1.hitung_total())

Produk.ganti_nama_toko("Glow Beauty")

print(Produk.cek_harga(35000))
```

Gambar 3.12 Pengujian Method